System Identification or System Modeling

Proakis/Manolakis Digital Signal Processing, Chapter 13

Unknown system called plant that we wish to identify

The system is modeled by an FIR filter with M adjustable coefficients

Both the plant and the model are excited by an input sequence $x[n]$. If $y[n]$ denotes the output of the plant and $\hat{y}[n]$ denotes the output of the model

$$\hat{y}[n] = \sum_{k=0}^{M-1} h[k]\,x[n-k]$$

We may form the error sequence

$$e[n] = y[n] - \hat{y}[n] = y[n] - \sum_{k=0}^{M-1} h[k]\,x[n-k]$$

and select the coefficients $h[k]$ to minimize cost function

$$C(h) = \sum_{n=0}^{N} \left[y[n] - \sum_{k=0}^{M-1} h[k]\,x[n-k] \right]^2 = \sum_{n=0}^{N} e^2[n]$$

where N+1 is the number of observations.

Partial derivative of $C(h)$ wrt a particular coefficient $h[m]$

$$\frac{\partial C(h)}{\partial h[m]} = \sum_{n=0}^{N} 2e[n] \underbrace{\frac{\partial e[n]}{\partial h[m]}}_{=-x[n-m]} = -2\sum_{n=0}^{N} e[n]\,x[n-m]$$

$$= -2\sum_{n=0}^{N} \left[y[n] - \sum_{k=0}^{M-1} h[k]\,x[n-k] \right] x[n-m] \qquad \text{for } m = 0, 1, \ldots M-1$$

In vector form, if

$$\underline{x}_n = \begin{bmatrix} x[n] \\ x[n-1] \\ \vdots \\ x[n-M+1] \end{bmatrix} \quad \text{and} \quad \underline{h} = \begin{bmatrix} h[0] \\ h[1] \\ \vdots \\ h[M-1] \end{bmatrix}$$

and

$$\hat{y}[n] = \underline{h}^T \cdot \underline{x}_n = [h[0], h[1], \dots, h[M-1]] \begin{bmatrix} x[n] \\ x[n-1] \\ \vdots \\ x[n-M+1] \end{bmatrix}$$

then $$C(\underline{h}) = \sum_{n=0}^{N} \left(y[n] - \underline{h}^T \underline{x}_n \right)^2$$

and the gradient is

$$\nabla_{\underline{h}} C(\underline{h}) = -2 \sum_{n=0}^{N} \underline{x}_n \left(y[n] - \underline{h}^T \underline{x}_n \right)$$

Note $$\nabla_{\underline{h}} C(\underline{h}) = \left[\frac{\partial C}{\partial h[0]}, \frac{\partial C}{\partial h[1]}, \dots, \frac{\partial C}{\partial h[M-1]} \right]^T$$

Generic n, $M = 3$

$$\begin{bmatrix} \frac{\partial C}{\partial h[0]} \\ \frac{\partial C}{\partial h[1]} \\ \frac{\partial C}{\partial h[2]} \end{bmatrix} = -2 \cdot \begin{bmatrix} x[n] \\ x[n-1] \\ x[n-2] \end{bmatrix} \cdot \left(y[n] - [h[0], h[1], h[2]] \cdot \begin{bmatrix} x[n] \\ x[n-1] \\ x[n-2] \end{bmatrix} \right)$$

Gradient descent, coefficient vector update

$$\underline{h}^{(t+1)} = \underline{h}^{(t)} - \mu \nabla_{\underline{h}} C(\underline{h}) = \underline{h}^{(t)} + 2\mu \sum_{n=0}^{N} \underline{x}_n \left(y[n] - \underline{h}^{(t)T} \underline{x}_n \right)$$

For an individual coefficient $h[m]$,

$$h^{(t+1)}[m] = h^{(t)}[m] + 2\mu \sum_{n=0}^{N} \left[y[n] - \sum_{k=0}^{M-1} h^{(t)}[k] x[n-k] \right] x[n-m]$$

This version uses the entire dataset ($N+1$ $x[n]$, $y[n]$ pairs) to compute one gradient update, that is batch gradient descent.

For an adaptive filter such as LMS (Least Mean Square), we usually use just the current sample n as an instantaneous estimate of the gradient:

$$\nabla_{\underline{h}} C_n(\underline{h}) = -2e[n]\underline{x}_n$$

Then

$$\underline{h}[n+1] = \underline{h}[n] + 2\mu e[n]\underline{x}_n$$

where

$$e[n] = y[n] - \underline{h}^T[n]\underline{x}_n$$

Often the factor of 2 is absorbed into the step size μ, giving the familiar LMS update

$$\underline{h}[n+1] = \underline{h}[n] + \mu e[n]\underline{x}_n$$

Thus, LMS uses stochastic gradient descent (SGD) where an immediate update is made for each new sample.

→ PyTorch SGD optimizes